UniBodget — Relazione di Progetto

Sistema Integrato di Finanza Personale, Budgeting e Monitoraggio Patrimoniale
Corso di Programmazione ad Oggetti — A.A. 2025/2026

Favour Emuwahen (favour.emuwahen@studio.unibo.it)

Arianna Palmucci (arianna.palmucci@studio.unibo.it)

Daniele Zappalà (daniele.zappala@studio.unibo.it)

Ottobre 2026

Sommario

La presente relazione documenta l'analisi, la progettazione architetturale, lo sviluppo implementativo e la verifica del progetto software **UniBodget**, un'applicazione desktop modulare in Java finalizzata alla gestione completa delle finanze personali, della contabilità multi-asset, del monitoraggio di portafogli d'investimento e della conversione multivaluta in tempo reale.

Il documento è redatto in rigorosa aderenza alla struttura prescritta dalla *Meta-relazione per Programmazione ad Oggetti* (Prof. Danilo Pianini). In particolare, vengono analizzati e dettagliati i seguenti sottosistemi architetturali:

1. Il cruscotto unificato (*Dashboard*), la contabilità di cassa, il monitoraggio dinamico delle soglie di budget, l'analisi statistica mese-su-mese e il tracciamento dei prestiti tra conoscenti (*Favour Emuwahen*);

2. Il sottosistema del dominio valute (*fiat*, *crypto*, *stock*, *custom*), il motore polimorfico di conversione, l'infrastruttura di integrazione con API di mercato esterne con cache resiliente, il modulo grafico del convertitore con simulatore bancario e la gestione trasversale di preferenze e temi dinamici (*Arianna Palmucci*);

3. Il monitoraggio del portafoglio titoli, l'esecuzione di ordini di compravendita e il calcolo del profitto/perdita non realizzato (*Daniele Zappalà*).

Indice

1 Analisi **3**
- 1.1 Descrizione e Requisiti . . . 3
- 1.2 Modello del Dominio . . . 6

2 Design **8**
- 2.1 Architettura . . . 8
- 2.2 Design Dettagliato (Arianna Palmucci) . . . 9
  - 2.2.1 Sistema Polimorfico delle Valute e Motore di Conversione . . . 9
  - 2.2.2 Architettura dei Provider di Cambio con Caching Resiliente . . . 10
  - 2.2.3 Monitoraggio dei Tassi, Allerte di Mercato e Watchlist Reattiva . . . 11
  - 2.2.4 Gestione delle Preferenze Utente, Temi Dinamici e Memento dello Stato . . . 11
  - 2.2.5 Composizione e Isolamento della View del Convertitore . . . 11
- 2.3 Design Dettagliato (Favour Emuwahen) . . . 12
  - 2.3.1 Coordinamento e Disaccoppiamento della Dashboard (Facade e Passive View) . . . 12
  - 2.3.2 Costruzione Fluente e Validazione delle Transazioni (Builder e Factory Pattern) . . . 14
  - 2.3.3 Monitoraggio Dinamico e Strategie di Allerta del Budget (Strategy Pattern) . . . 15
  - 2.3.4 Algoritmo di Analisi Comparativa Mensile (Template Method Pattern) 17
  - 2.3.5 Filtraggio Composito ed Estendibile della Cronologia (Specification Pattern) . . . 18

3 Sviluppo **20**
- 3.1 Testing Automatizzato . . . 20
- 3.2 Note di Sviluppo . . . 21
  - 3.2.1 Favour Emuwahen . . . 21
  - 3.2.2 Arianna Palmucci . . . 23

4 Commenti finali **25**
- 4.1 Autovalutazione e lavori futuri . . . 25
  - 4.1.1 Favour Emuwahen . . . 25
  - 4.1.2 Arianna Palmucci . . . 26
- 4.2 Difficoltà Incontrate e Commenti per i Docenti . . . 27

A Guida Utente **28**

Capitolo 1

Analisi

In questo capitolo vengono definiti il contesto operativo, gli obiettivi e i requisiti individuati nella fase preliminare di studio del problema, astraendo interamente dalle tecnologie e dai dettagli implementativi del software. Viene inoltre formalizzato il modello concettuale del dominio applicativo.

1.1 Descrizione e Requisiti

Contesto e Motivazione

Negli ultimi anni, la gestione del patrimonio economico individuale ha subito una profonda trasformazione: l'utente contemporaneo non si limita più alla gestione di un singolo conto corrente bancario in valuta locale, ma detiene frequentemente risorse frammentate su più canali, tra cui conti di pagamento digitali, contante fisico, portafogli di criptovalute decentralizzate e conti deposito o broker d'investimento per la compravendita di titoli azionari internazionali.

Questo scenario multi-asset genera una serie di problematiche concrete:

1. **Frammentazione e Difficoltà di Aggregazione:** Monitorare il valore reale complessivo delle proprie disponibilità richiede di accedere a piattaforme separate con interfacce, unità di conto e logiche operative totalmente eterogenee.

2. **Inadeguatezza degli Strumenti Tradizionali:** L'uso di fogli di calcolo elettronici (es. Excel o Google Sheets) si rivela tedioso, incline a errori di trascrizione manuale e incapace di reagire tempestivamente alla volatilità dei tassi di cambio.

3. **Opacità dei Costi di Conversione Bancaria:** Quando si effettuano transazioni o trasferimenti transfrontalieri, gli istituti di credito tradizionali applicano spesso commissioni fisse e maggiorazioni sul tasso di cambio (*spread*) poco trasparenti, rendendo difficile per l'utente valutare l'effettiva onerosità del cambio.

4. **Perdita di Sovranità sul Dato:** Molti applicativi commerciali per la finanza personale richiedono la condivisione obbligatoria delle credenziali bancarie su server cloud di terze parti, sollevando rilevanti preoccupazioni in termini di riservatezza e sicurezza informatica.

Obiettivi del Sistema

UniBodget è stato progettato per colmare queste lacune, proponendosi come un cruscotto desktop integrato, autonomo e focalizzato sulla massima precisione, trasparenza ed ergonomia d'uso. Gli obiettivi cardine del sistema comprendono:

- **Trattamento Omogeneo del Multi-Asset:** Fornire un'astrazione contabile uniforme capace di gestire valute a corso legale (*fiat*), asset crittografici (*crypto*) e titoli azionari (*stock*), garantendo per ciascuna categoria la rappresentazione decimale esatta e la corretta formattazione simbolica.
- **Separazione tra Cassa e Investimenti:** Consentire all'utente di isolare la liquidità destinata alle spese ordinarie quotidiane (`CashAccount`) dal patrimonio vincolato in asset di investimento speculativi o a lungo termine (`InvestmentAccount`), ciascuno con metriche dedicate di monitoraggio (budget di spesa vs. profitto/perdita non realizzato).
- **Trasparenza di Mercato in Tempo Reale:** Interfacciarsi con molteplici provider di quotazioni pubbliche per ottenere tassi di cambio costantemente sincronizzati, offrendo un convertitore istantaneo bidirezionale, un simulatore comparativo delle commissioni bancarie e grafici storici sull'andamento dei prezzi.
- **Monitoraggio Proattivo:** Permettere la configurazione di soglie limite personalizzate sui cambi, notificando tempestivamente l'utente all'innesco delle condizioni stabilite.
- **Personalizzazione Ergonomica Completa:** Adattare l'esperienza visiva alle preferenze dell'utente mediante commutazione dinamica a caldo tra tema scuro (*Dark*) e tema chiaro (*Light*), selezione delle famiglie tipografiche, dimensionamento flessibile dei font e memorizzazione di uno storico degli stati di configurazione (*Memento*).
- **Persistenza Locale e Decentralizzata:** Assicurare la completa privacy e indipendenza operativa attraverso il salvataggio automatico su file JSON strutturati e versionabili, con strategie di auto-ripristino trasparente in caso di anomalie dei file.

Requisiti Funzionali

I requisiti funzionali del sistema sono articolati nelle seguenti aree:

1. **Gestione Portafogli e Contabilità di Cassa (Competenza - Favour Emuwahen):**
    - Visualizzazione della Dashboard unificata contenente saldo del conto corrente attivo, controvalore globale, grafici di ripartizione e breakdown spese, indicatore di budget, prestiti e transazioni recenti.
    - Creazione, personalizzazione, ridenominazione ed eliminazione di portafogli monetari (*fiat*).
    - Registrazione, modifica ed eliminazione di transazioni categorizzate (spese ordinarie, entrate, prestiti tra privati), ciascuna caratterizzata da importo, valuta, data, categoria e note.
    - Monitoraggio dinamico del budget mensile con definizione di tetti massimi di spesa e soglie di allerta percentuali (*Safe*, *Warning*, *Critical*).

- Gestione e tracciamento dei prestiti informali tra conoscenti (*Friend Loans*), con calcolo del debito/credito residuo aperto e registrazione delle restituzioni.
- Calcolo statistico comparativo dei trend mensili (*Monthly Insights*) mese su mese per uscite, entrate e risparmio netto.
- Ricerca e filtraggio avanzato dello storico transazioni per parola chiave, categoria, tipo operazione, intervallo temporale e ordinamento dinamico.
- Esportazione della cronologia dei movimenti in formato tabellare conforme allo standard CSV RFC 4180.

2. **Monitoraggio Portafogli Titoli e Investimenti (Competenza - Daniele Zappalà):**
   - Creazione e gestione di conti dedicati ad asset d'investimento (azioni e criptovalute).
   - Esecuzione di ordini di acquisto e vendita con registrazione del prezzo unitario di mercato, delle commissioni di transazione e calcolo automatico del profitto/perdita (*unrealized e realized profit/loss*).
   - Valutazione complessiva del portafoglio titoli convertito nella valuta di riferimento dell'utente.
3. **Conversione Valute e Monitoraggio Mercati (Competenza - Arianna Palmucci):**
   - Conversione istantanea bidirezionale tra qualsiasi coppia di valute censite (*fiat*, criptovalute, titoli azionari) mantenendo precisione contabile senza troncamenti indebiti.
   - Simulatore del cambio bancario: comparazione dei costi di conversione applicati dai principali istituti di credito, con esplicitazione dello spread percentuale e delle commissioni fisse applicate.
   - Consultazione dello storico tassi con generazione di grafici temporali per evidenziare l'andamento del cambio nel tempo.
   - Lista di osservazione (*Watchlist*): selezione rapida e salvataggio delle coppie valutarie preferite per una consultazione immediata dei tassi correnti.
   - Notifiche e allerte sul cambio: definizione di soglie di allarme e avviso interattivo all'utente al verificarsi della condizione impostata.
4. **Configurazione, Temi e Persistenza (Competenza - Arianna Palmucci):**
   - Scelta della valuta base applicativa globale, con ricalcolo immediato dei totali patrimoniali aggregati.
   - Modifica dinamica del tema grafico (*Dark*, *Light*) e scelta del font per i caratteri con applicazione immediata su tutte le viste senza riavviare l'applicazione.
   - Memorizzazione della cronologia degli stati di configurazione (*Memento/Snapshot*) per la visualizzazione e il ripristino di impostazioni precedenti.
   - Salvataggio atomico automatico dello stato applicativo su file JSON strutturati.

Requisiti Non Funzionali

1. **Robustezza e Continuità di Servizio:** L'applicazione deve garantire il proprio funzionamento anche in assenza temporanea di connettività Internet o in caso di rate-limiting da parte delle API esterne, ricorrendo a cache trasparente e dati simulati di fallback.

2. **Precisione Numerica:** I calcoli contabili, i bilanci e le conversioni monetarie devono azzerare gli errori di approssimazione in virgola mobile binaria, operando con aritmetica decimale esatta ad alta precisione (`BigDecimal`).

3. **Modularità e Separazione delle Responsabilità:** L'architettura deve aderire al pattern Model-View-Controller, garantendo la totale indipendenza del modello di dominio dal framework grafico JavaFX.

4. **Qualità del Codice e Manutenibilità:** Il codice deve rispettare standard rigorosi di static analysis (Checkstyle, PMD, SpotBugs) senza deroghe o soppressioni ingiustificate, privilegiando immutabilità, information hiding e copie difensive.

5. **Portabilità Desktop:** Completa compatibilità multipiattaforma con ambienti Java 21 LTS su sistemi operativi Windows, macOS e Linux.

1.2 Modello del Dominio

Il modello concettuale del dominio astrae le entità del problema finanziario prima di qualsiasi scelta tecnologica o implementativa.

L'elemento fondamentale del sistema è il **Wallet**, che rappresenta un portafoglio finanziario in grado di calcolare il proprio saldo complessivo sommando algebricamente la propria collezione ordinata di movimenti (**Historical**). Ciascun movimento è rappresentato dall'entità concettuale **Transaction**, caratterizzata da una marca temporale, una descrizione e un **Asset**.

L'entità **Asset** unisce l'aspetto quantitativo (importo numerico ad alta precisione) all'unità di conto monetaria (**CurrencyUnit**). All'interno del dominio contante, la **Cash-Transaction** specializza il movimento specificando una **Category** (caratterizzata da un nome e da una tipologia semantica: entrata, spesa, prestito) e gli estremi di soggetti terzi nel caso di crediti/debiti tra conoscenti (*Friend Loan*). Parallelamente, il contesto temporale di spesa è governato dal **BudgetSettings**, che definisce il limite massimo periodico e la soglia di pre-allarme, mentre l'entità concettuale **BudgetStatus** riflette la condizione risultante del consumo del capitale.

Le entità principali del dominio comprendono:

- **CurrencyUnit:** Rappresenta un'unità monetaria o finanziaria astratta, specializzata in valuta a corso legale (`FiatCurrency`), valuta crittografica (`CryptoCurrency`), asset azionario (`StockMarketCurrency`) o valuta personalizzata (`Currency`).

- **Asset:** Valore patrimoniale quantificato, composto da un importo numerico decimale esatto e dalla relativa unità di misura monetaria (`CurrencyUnit`).

- **ExchangeRate:** Rappresentazione della quotazione di mercato relativa a una coppia ordinata di valute in un determinato istante o data.

- **CurrencyAlert:** Regola di monitoraggio che lega una coppia di valute a una soglia numerica limite.

- **WatchList:** Insieme ordinato di coppie valutarie preferite dall'utente per la consultazione rapida.
- **Bank:** Modello di intermediario finanziario che associa a una specifica valuta uno spread percentuale e una commissione fissa di cambio.
- **Wallet:** Entità astratta che rappresenta un contenitore patrimoniale, specializzata in conto monetario (`CashAccount`) e conto investimenti (`InvestmentAccount`).
- **Transaction:** Registrazione contabile associata a un portafoglio, specializzata in transazione di cassa (`CashTransaction`) e ordine di borsa (`InvestmentTransaction`).
- **Category:** Classificazione semantica di una transazione (Spesa, Entrata, Prestito) con relativa palette cromatica identificativa.
- **BudgetSettings:** Parametri di bilancio configurati dall'utente, comprendenti il limite mensile di spesa e la percentuale di pre-allarme.
- **BudgetStatus:** Istantanea valutativa dello stato del budget (*Safe*, *Warning*, *Exceeded*).
- **FriendLoanSummary:** Aggregazione contabile della situazione debitoria e creditoria verso un determinato conoscente.
- **Settings:** Insieme delle configurazioni globali dell'utente (valuta di riferimento, tema visivo, dimensioni della finestra).

In Figura 1.1 viene riportato lo schema concettuale delle entità del dominio e delle loro relazioni.

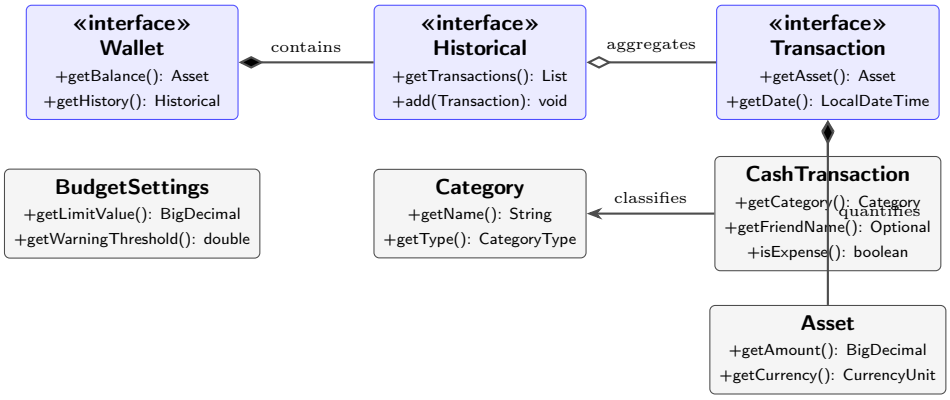


Figura 1.1: Modello concettuale del dominio: entità e relazioni contabili fondamentali.

Capitolo 2

Design

In questo capitolo vengono descritte la struttura architetturale complessiva di UniBodget e le strategie di progettazione dettagliate adottate dai componenti del gruppo per la risoluzione dei problemi cardine a ciascuno assegnati.

2.1 Architettura

Il sistema adotta il pattern architetturale **Model-View-Controller (MVC)** integrato con una struttura reattiva basata su un bus di messaggi disaccoppiato (`MessageBus`). Tale impostazione garantisce il completo isolamento della logica applicativa rispetto al toolkit grafico (JavaFX).

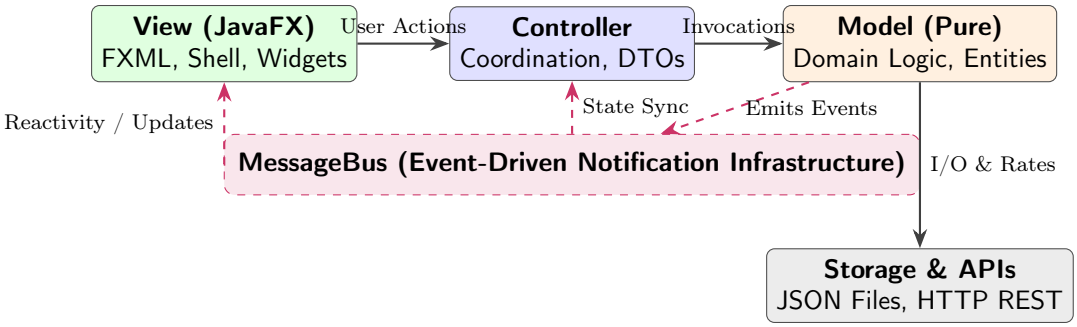


Figura 2.1: Rappresentazione ad alto livello dell'architettura MVC e del MessageBus disaccoppiato.

Ruoli dei Componenti Architetturali

- **Model:** Incapsula lo stato persistente dell'applicazione, le regole di business contabili, gli algoritmi di conversione e i meccanismi di serializzazione. Il modello è puro: non contiene alcuna dipendenza verso librerie grafiche (`javafx.*`).
- **Controller:** Intercetta i comandi dell'utente inviati dalla vista, coordina le invocazioni verso i servizi del modello e trasforma le entità di business in strutture dati di presentazione (view models e records immutabili) consumabili dalla GUI.
- **View:** Costruita in JavaFX attraverso una combinazione di file FXML e viste programmatiche modulari. Si occupa del rendering grafico e dell'ascolto delle interazioni utente, delegando ogni logica decisionale al rispettivo controller.

Disaccoppiamento View-Controller

Il disaccoppiamento è garantito dall'adozione del principio di inversione delle dipendenze:

- I controller non detengono riferimenti a classi di layout o nodi visivi specifici (non manipolano direttamente `TextField`, `TableView` o controlli FXML), ma operano tramite interfacce di callback e listener funzionali (`Consumer`, `Runnable`, `Supplier`).
- Nel modulo di conversione (`CurrencyConverterModule`) e nel modulo dashboard (`DashboardModule`), l'intera vista è esposta verso la shell principale come un generico nodo radice (`Parent` o `Region`), rendendo la composizione dell'interfaccia totalmente agnostica rispetto ai widget interni.
- La sostituzione completa del framework grafico (ad esempio verso Swing o una web UI) richiederebbe unicamente la riscrittura del layer View, lasciando immutati sia il Controller sia l'intero Model.

2.2 Design Dettagliato (Arianna Palmucci)

In questa sezione vengono dettagliati i cinque principali problemi di progettazione affrontati e risolti da Arianna Palmucci, con l'analisi delle alternative scartate, la descrizione delle soluzioni e la reificazione dei design pattern adottati.

2.2.1 Sistema Polimorfico delle Valute e Motore di Conversione

1. Descrizione del problema

L'applicazione necessita di operare con classi di attività finanziarie eterogenee (valute a corso legale, criptovalute e titoli azionari). Tali attività presentano caratteristiche intrinseche profondamente disomogenee:

- Le valute fiat operano con convenzioni a 2 o 4 decimali e identificatori ISO standard (EUR, USD, GBP, ecc.).
- Le criptovalute presentano frazionabilità elevatissima (fino a 8–18 decimali) e richiedono identificatori specifici per le API di quotazione (es. `apiId` come `"bitcoin"` o `"ethereum"`).
- I titoli azionari operano con ticker di borsa (es. `AAPL`, `GOOGL`) e mercati di riferimento regolamentati.

Inoltre, il calcolo della conversione tra due qualsiasi valute deve essere flessibile, estendibile e svincolato dall'implementazione specifica della sorgente dei tassi.

2. Soluzione proposta e alternative

- **Alternativa scartata (Classe monolitica o enum universale):** Raggruppare tutte le tipologie in un'unica enum con campi facoltativi avrebbe violato i principi *Open/Closed* (OCP) e *Single Responsibility* (SRP), obbligando a modificare codice esistente per aggiungere nuove valute o mercati.
- **Soluzione adottata:** Definizione dell'interfaccia comune `CurrencyUnit`, implementata in modo polimorfico da enum fortemente tipizzate: `FiatCurrency`, `CryptoCurrency`

e `StockMarketCurrency`, oltre alla classe aperta `Currency` per asset personalizzati. Per l'esecuzione dei cambi è stata definita l'interfaccia `CurrencyConverter`, la cui implementazione principale `BasicCurrencyConverter` esegue calcoli rigorosi su `BigDecimal` con rounding bancario (`RoundingMode.HALF_UP`), incapsulando il risultato nel record immutabile `CurrencyConversionResult`.

3. Schema UML e Interazioni

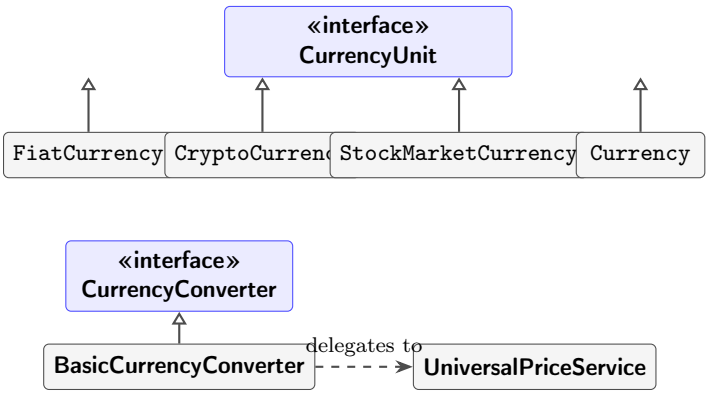


Figura 2.2: Diagramma delle classi: gerarchia polimorfica di `CurrencyUnit` e motore di conversione.

4. Pattern utilizzati e reificazione

- **Strategy Pattern:** L'interfaccia `CurrencyConverter` rappresenta la *Strategy* astratta per il calcolo delle conversioni. `BasicCurrencyConverter` costituisce la *Concrete Strategy* che esegue la triangolazione ponderata sui tassi correnti.
- **Adapter Pattern:** I dati eterogenei restituiti dai provider remoti vengono normalizzati attraverso `ExchangeRateAPIClient` e `UniversalPriceService`, che fungono da *Adapter* verso le strutture di dominio interne.

2.2.2 Architettura dei Provider di Cambio con Caching Resiliente

1. Descrizione del problema

I servizi esterni che forniscono le quotazioni di cambio impongono vincoli critici di rate limiting, latenza di rete e disponibilità disomogenea.

2. Soluzione proposta e alternative

- **Alternativa scartata (Chiamate dirette synchronous non memorizzate):** Interrogare la rete ad ogni richiesta utente porterebbe rapidamente all'esaurimento delle quote, al blocco dell'interfaccia e a crash in assenza di connessione.
- **Soluzione adottata:** Introduzione di una cache in memoria controllata a tempo (*TTL*) all'interno di `ExchangeRateAPIImpl` e `UniversalPriceService`, affiancata da un provider interamente autonomo e deterministico (`MockExchangeRateAPI`) per ambienti offline e test.

3. Pattern utilizzati e reificazione

- **Cache Proxy Pattern:** `ExchangeRateAPIImpl` controlla e protegge l'accesso alla risorsa remota costosa (chiamate HTTP), intercettando le richieste e soddisfacendole dalla cache interna quando ancora valide.
- **Null Object / Fallback Strategy Pattern:** `MockExchangeRateAPI` offre un'implementazione autonoma e affidabile dei contratti di mercato, garantendo continuità e determinismo.

2.2.3 Monitoraggio dei Tassi, Allerte di Mercato e Watchlist Reattiva

1. Descrizione del problema

Gestione delle coppie valutarie preferite e configurazione di soglie limite con avvisi grafici all'utente, senza contaminare il modello con classi grafiche.

2. Soluzione proposta e alternative

Separazione netta: `CurrencyAlert` (record immutabile), `CurrencyAlertService` (servizio puro di filtraggio), `MarketAlert` (mediatore grafico su thread JavaFX) e `WatchList` (insieme incapsulato e protetto).

3. Pattern utilizzati e reificazione

- **Observer Pattern:** `CurrencyAlertService` valuta le soglie a ogni nuova conversione e notifica i componenti visuali (`MarketAlert`).
- **Defensive Design:** `WatchList` restituisce collezioni immutabili (`Collections.unmodifiableSet`) per proteggere lo stato interno.

2.2.4 Gestione delle Preferenze Utente, Temi Dinamici e Memento dello Stato

1. Descrizione del problema

Supporto alla personalizzazione grafica con storicizzazione delle preferenze (Undo/Restore) e propagazione a caldo del cambio tema senza riavviare l'applicazione.

2. Soluzione proposta e alternative

Adozione del **Memento Pattern** (`Settings` come Originator, `SettingsSnapshot` come Memento, `SettingsManager` come Caretaker) e del registro reattivo `ThemeManager` per l'iniezione dinamica dei fogli di stile CSS sulle scene JavaFX registrate.

2.2.5 Composizione e Isolamento della View del Convertitore

1. Descrizione del problema

La sezione di conversione unisce quattro widget interattivi complessi (`ConverterWidgetFX`, `BankConverterWidgetFX`, `CurrencyHistoryChartView`, `WatchlistWidgetFX`). Era essenziale aggregarli disaccoppiandoli dalla shell principale.

2. Soluzione proposta e alternative

Adozione del **Facade Pattern** (`CurrencyConverterViewFX` unifica i widget interni) e del **Module Pattern** (`CurrencyConverterModule` incapsula ciclo di vita e controller, impiegando lazy initialization).

2.3 Design Dettagliato (Favour Emuwahen)

In questa sezione vengono documentate le decisioni architetturali, i pattern di progettazione e le motivazioni tecniche adottate nello sviluppo del sottosistema di mia competenza: la **Dashboard interattiva**, la contabilità dei **Portafogli di Cassa (*Cash Wallets*)**, la costruzione e validazione delle **Transazioni Contanti (*Cash Transactions*)**, l'analisi comparativa dei **Trend Mensili (*Monthly Insights*)**, il monitoraggio dinamico delle soglie di **Budget** e il **Filtraggio declarativo** dello storico movimenti.

2.3.1 Coordinamento e Disaccoppiamento della Dashboard (Facade e Passive View)

1. Descrizione del problema

La Dashboard rappresenta il centro operativo dell'applicazione: deve aggregare ed esporre simultaneamente una pluralità di informazioni eterogenee:

- Saldo totale e controvalore convertito nella valuta di riferimento globale;
- Ripartizione percentuale e scomposizione delle uscite per categoria;
- Stato di avanzamento del budget mensile con percentuale di consumo;
- Posizione debitoria/creditoria netta verso amici (*Friend Loans*);
- Indicatori di tendenza mese su mese (*Monthly Insights*);
- Cronologia dei movimenti recenti con filtri dinamici di ricerca.

Consentire al controller o alla vista di interrogare direttamente i singoli servizi di dominio (gestione conti, categorie, calcolo budget, prestiti, statistiche) avrebbe introdotto un accoppiamento elevatissimo (*tight coupling*), violando la Legge di Demetra e rendendo la View dipendente dai dettagli interni del Model. Inoltre, esporre entità mutabili del dominio alla GUI avrebbe compromesso l'integrità contabile.

2. Soluzione proposta e alternative

- **Alternativa scartata (Controller multi-servizio accoppiato):** Far dialogare il controller direttamente con ciascun servizio avrebbe portato alla creazione di una *God Class* con decine di dipendenze dirette, rendendo complessa la manutenzione e impossibili i test in isolamento.
- **Alternativa scartata (Passaggio di entità di dominio alla View):** Far consumare direttamente `CashAccount` e `Category` ai componenti JavaFX espone la rappresentazione interna a manipolazioni esterne non controllate (*representation exposure*).

- **Soluzione adottata:** Introduzione del pattern **Facade** tramite l'interfaccia `DashboardFacade` e la classe `DefaultDashboardFacade`. Il controller invoca un unico metodo ad alto livello (`loadDashboard()`), che orchestra i sotto-servizi e restituisce un oggetto istantanea immutabile: `DashboardSnapshot`. Il controller trasforma quindi tale snapshot in una gerarchia di record immutabili di presentazione (`DashboardViewState`), consegnati alla vista passiva (`DefaultDashboardView`) per il solo rendering grafico.

3. Schema UML e Interazioni

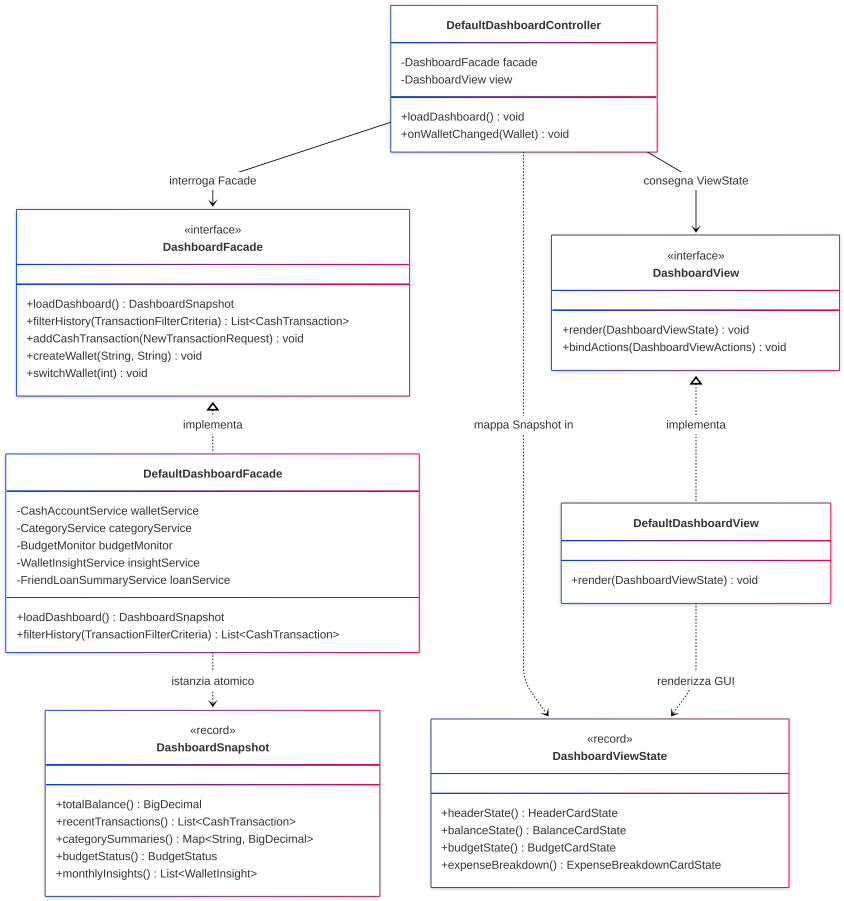


Figura 2.3: Reificazione del Facade Pattern e della Passive View per il sottosistema Dashboard.

4. Pattern utilizzati e reificazione

- **Facade Pattern:** `DefaultDashboardFacade` agisce come facciata unificata, incapsulando `CashAccountService`, `CategoryService`, `BudgetMonitor`, `FriendLoanSummaryService` e `WalletInsightService`.
- **Value Object / Snapshot Pattern:** `DashboardSnapshot` raccoglie i dati aggregati del modello all'istante di campionamento, garantendo consistenza atomica.
- **Passive View (MVC):** `DefaultDashboardView` non accede mai al modello; riceve lo stato `DashboardViewState` dal controller e notifica le interazioni utente tramite l'interfaccia `DashboardViewActions`.

- **Observer Pattern:** `DefaultDashboardController` implementa `WalletObserver`; quando il servizio portafogli notifica un cambiamento, il controller interroga la facciata e aggiorna reattivamente la vista.

2.3.2 Costruzione Fluente e Validazione delle Transazioni (Builder e Factory Pattern)

1. Descrizione del problema

Nel contesto della contabilità di cassa, una transazione monetaria (`CashTransaction`) presenta una marcata eterogeneità di parametri:

- **Attributi obbligatori:** identificativo univoco (`UUID`), asset monetario con importo e valuta (`Asset`), categoria contabile (`Category`), data di registrazione (`LocalDate`) e causale;
- **Attributi contestuali/opzionali:** note aggiuntive descrittive, nominativo dell'amico (`friendName`) e identificativo del prestito correlato (`friendLoanId`) nel caso di prestiti fiduciari.

L'approccio tradizionale basato su costruttori sovraccaricati (*telescoping constructors*) generava confusione e rischio di inversione di argomenti del medesimo tipo (stringhe). D'altro canto, consentire l'uso di *setter* avrebbe reso l'oggetto mutabile, violando l'integrità contabile. Era inoltre necessario automatizzare la corretta assegnazione del segno algebrico degli importi (es. importi negativi per spese e prestiti erogati, positivi per entrate e restituzioni).

2. Soluzione proposta e alternative

- **Alternativa scartata (Costruttori multipli telescopici):** Impraticabile con l'aumentare dei campi; obbliga a passare parametri `null` per i valori facoltativi.
- **Alternativa scartata (JavaBeans mutabili):** Scartata categoricamente; in un'applicazione contabile, permettere la modifica di data, importo o categoria post-costruzione espone a corruzioni dello storico.
- **Soluzione adottata:** Implementazione del pattern **Builder** come classe interna statica fluente (`CashTransaction.Builder`), coadiuvata dalla classe specializzata **Factory** (`CashTransactionFactory`). Il builder valida rigorosamente le invarianti nel metodo terminale `build()` (es. vincolo di presenza obbligatoria di `friendName` se la transazione rappresenta un prestito), garantendo che nessuna transazione possa esistere in uno stato incoerente.

3. Schema UML e Interazioni

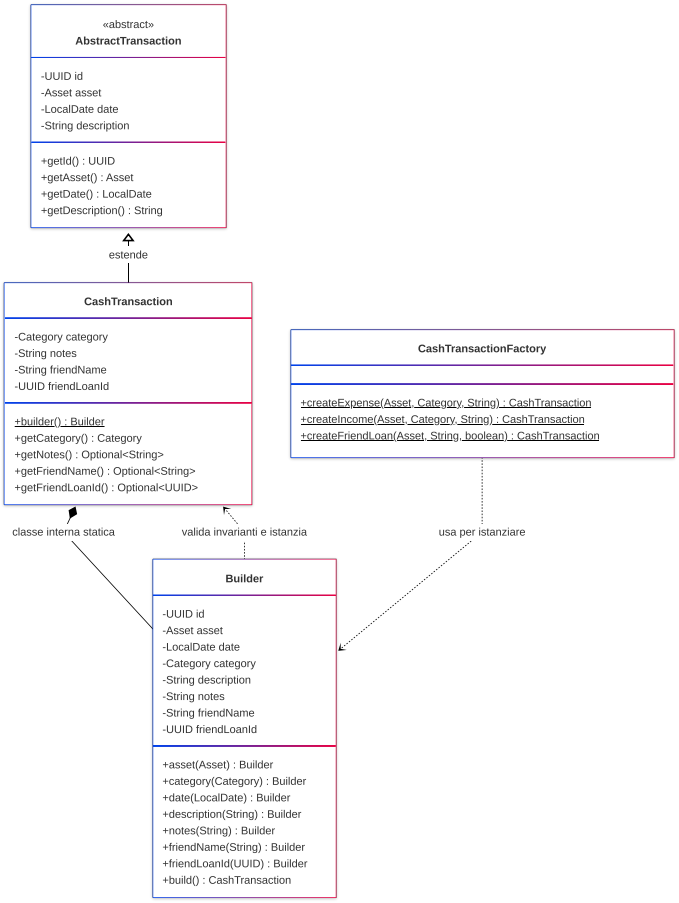


Figura 2.4: Reificazione dei pattern Builder e Factory per la creazione sicura di `CashTransaction`.

4. Pattern utilizzati e reificazione

- **Builder Pattern:** `CashTransaction.Builder` consente la composizione fluente passo-passo dei parametri, validando le invarianti di business prima di restituire l'oggetto immutabile.
- **Factory Pattern:** `CashTransactionFactory` incapsula la costruzione dei casi d'uso canonici (`createExpense`, `createIncome`, `createFriendLoan`), gestendo in automatico la polarità degli importi (`Asset.amount().negate()`) e preservando il principio di singola responsabilità (SRP).

2.3.3 Monitoraggio Dinamico e Strategie di Allerta del Budget (Strategy Pattern)

1. Descrizione del problema

Il controllo delle spese rispetto al budget mensile richiede di valutare la spesa cumulativa corrente rispetto al tetto limite prefissato dall'utente e a una soglia percentuale di avviso (es. 80%). In base al consumo calcolato, il sistema deve emettere un responso qualitativo differenziato (`SAFE`, `WARNING`, `EXCEEDED`), corredato da messaggi contestuali e badge

cromatici di allerta nella dashboard. La logica di valutazione doveva essere totalmente estendibile, consentendo l'introduzione futura di nuovi criteri di allarme (es. proiezioni di spesa predittiva a metà mese o budget per singola categoria) senza dover modificare il monitor centrale.

2. Soluzione proposta e alternative

- **Alternativa scartata (Istruzioni condizionali annidate nel monitor):** L'uso di ramificazioni `if-else` o `switch` cablate all'interno di `DefaultBudgetMonitor` avrebbe violato il principio *Open/Closed* (OCP), imponendo la riscrittura del codice collaudato a ogni variazione delle regole di calcolo.
- **Soluzione adottata:** Adozione dello **Strategy Pattern**. L'interfaccia `BudgetAlertStrategy` definisce il contratto formale per la valutazione algoritmica della spesa. Il monitor (`DefaultBudgetMonitor`) delega la valutazione alla strategia appropriata, ottenendo come risultato il record immutabile `BudgetStatus`.

3. Schema UML e Interazioni

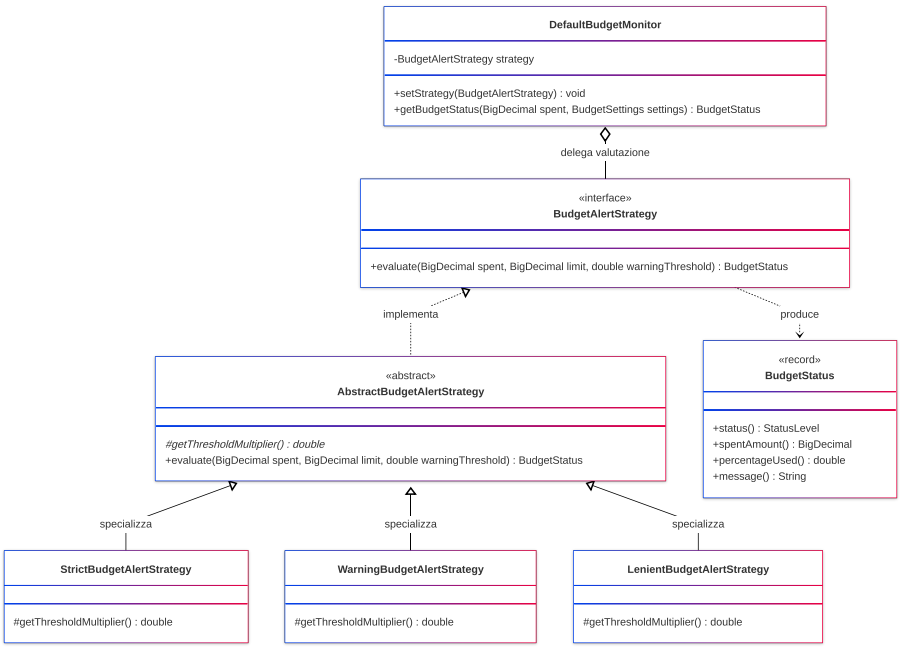


Figura 2.5: Reificazione dello Strategy Pattern per la valutazione delle soglie di allarme del Budget.

4. Pattern utilizzati e reificazione

- **Strategy Pattern:** `BudgetAlertStrategy` costituisce la *Strategy* astratta; `SafeBudgetStrategy`, `WarningBudgetStrategy` e `CriticalBudgetStrategy` rappresentano le *Concrete Strategies*. Il client (`DefaultBudgetMonitor`) agisce da *Context*, mantenendo completa indipendenza dalle specifiche condizioni di soglia.
- **Open/Closed Principle (OCP):** È possibile introdurre nuove strategie di allerta creando nuove classi che implementano l'interfaccia, senza alterare il codice esistente del monitor.

2.3.4 Algoritmo di Analisi Comparativa Mensile (Template Method Pattern)

1. Descrizione del problema

La dashboard include schede sintetiche per l'analisi comparativa mese-su-mese (*Monthly Insights*) relative a tre indicatori finanziari fondamentali:

1. Variazione complessiva delle uscite (*Spending*);
2. Variazione complessiva delle entrate (*Income*);
3. Tasso e volume di risparmio netto (*Net Savings*).

Ciascuno di questi calcoli condivide la medesima struttura algoritmica:

1. Partizionare lo storico transazioni isolando il mese corrente e il mese solare antecedente;
2. Aggregare la metrica monetaria di interesse per ciascuno dei due periodi;
3. Calcolare la variazione percentuale (*delta percentage*), gestendo i casi limite di divisione per zero quando il mese precedente ha valore nullo;
4. Determinare la qualità semantica del trend (`POSITIVE`, `NEGATIVE`, `NEUTRAL`);
5. Istanziare il record `WalletInsight` con testo formattato e badge di tendenza.

Duplicare questa sequenza logica in tre servizi distinti avrebbe generato ridondanza di codice e rischi di calcolo difforme.

2. Soluzione proposta e alternative

- **Alternativa scartata (Metodo monolitico procedurale):** Raggruppare i tre calcoli in un unico blocco condizionale avrebbe creato un metodo voluminoso, fragile e difficilmente manutenibile.
- **Soluzione adottata:** Adozione del pattern **Template Method**. La classe astratta `AbstractMonthlyInsightTemplate` fissa lo scheletro invariante dell'algoritmo nel metodo `generateInsight()`, delegando alle sottoclassi concrete la sola implementazione dei metodi *hook* primitivi: `computeMetric(transactions)` e `determineTrend(delta, current, previous)`.

3. Schema UML e Interazioni

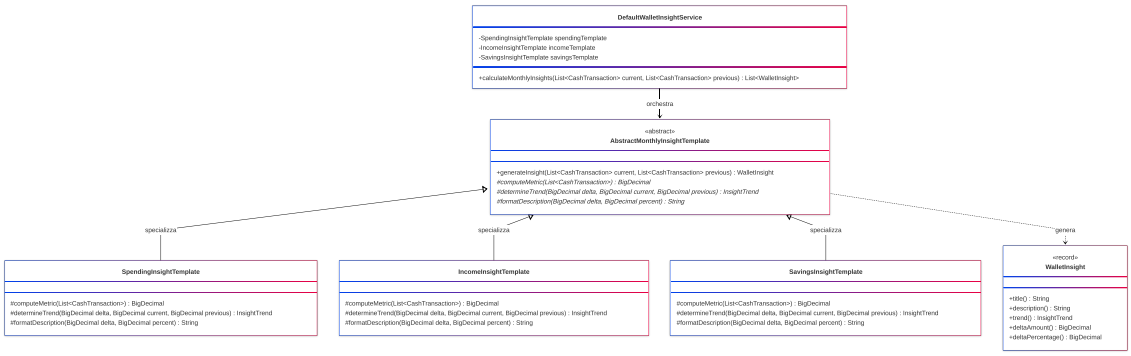


Figura 2.6: Reificazione del Template Method Pattern per il calcolo comparativo dei Monthly Insights.

4. Pattern utilizzati e reificazione

- **Template Method Pattern:** Il metodo `generateInsight()` di `AbstractMonthlyInsightTemplate` definisce l'ordine di esecuzione immutabile (filtraggio → calcolo metriche → delta percentuale con gestione divisione per zero → determinazione trend).
- **Inversione del Controllo (*Hollywood Principle*):** Le classi concrete (`SpendingInsightTemplate`, `IncomeInsightTemplate`, `SavingsInsightTemplate`) non controllano il flusso; implementano solo la specifica metrica contabile e la semantica del trend (ad esempio, per le uscite un delta di spesa negativo è un trend `POSITIVE`, mentre per le entrate è considerato `NEGATIVE`).

2.3.5 Filtraggio Composito ed Estendibile della Cronologia (Specification Pattern)

1. Descrizione del problema

La tabella dei movimenti finanziari della Dashboard mette a disposizione dell'utente una barra di ricerca avanzata con criteri di filtraggio combinabili a piacere:

- Intervallo temporale delimitato da data iniziale (*from date*) e finale (*to date*);
- Tipologia contabile del movimento (*Tutte, Solo Entrate, Solo Spese, Prestiti*);
- Categoria di spesa o entrata selezionata dal menu a tendina;
- Ricerca testuale libera per parola chiave su causale e note descrittive;
- Criterio di ordinamento dinamico (cronologico crescente/decrescente o per importo).

Una gestione procedurale basata su controlli condizionali concatenati (`if-else` o cicli mutabili) avrebbe innescato un'esplosione combinatoria ingestibile ($2^4 = 16$ possibili rami logici), rendendo il codice fragile, prono a regressioni e ostico da collaudare nelle singole combinazioni. Inoltre, l'aggiunta di un nuovo parametro di ricerca (ad esempio per importo minimo/massimo) avrebbe costretto a riscrivere la logica interna dell'intero servizio.

2. Soluzione proposta e alternative

- **Alternativa scartata (Filtraggio procedurale con if annidati):** Scartata per la natura non componibile e l'elevata fragilità del flusso imperativo.
- **Alternativa scartata (Predicati Java standard non incapsulati):** L'uso di semplici predicati anonimi (`Predicate<CashTransaction>`) dispersi nel controller avrebbe privato il dominio di astrazioni formali nominative e riusabili.
- **Soluzione adottata:** Adozione dello **Specification Pattern** di derivazione *Domain-Driven Design*. L'interfaccia formale `TransactionSpecification` estende `Predicate<CashTransaction` e implementa metodi di default componibili algebricamente (`and`, `or`, `not`). Attorno a questa radice polimorfica sono state reificate le specifiche atomiche (`CategoryNameSpecification`, `CategoryTypeSpecification`, `DateRangeSpecification`, `KeywordSpecification`), assemblate in modo compatto dalla factory `TransactionSpecifications.fromCriteria(criteria)`. Il servizio `DefaultTransactionHistoryFilterService` riceve i criteri, recupera la specifica composta e filtra lo stream delle transazioni in un singolo passaggio dichiarativo.

3. Schema UML e Interazioni

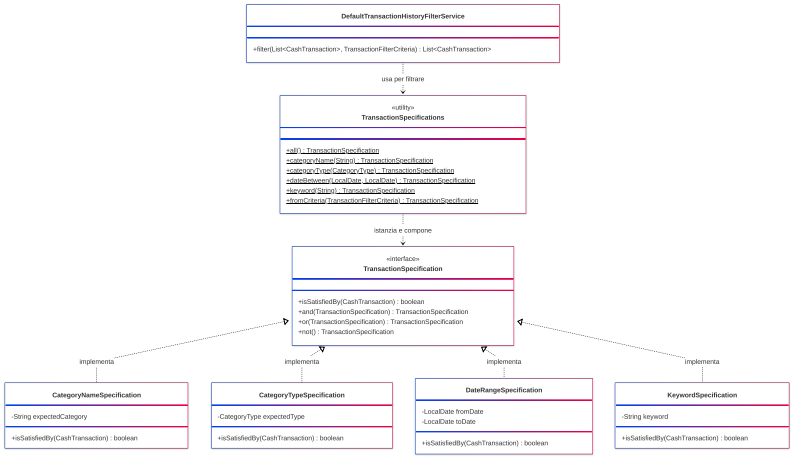


Figura 2.7: Reificazione dello Specification Pattern per il filtraggio avanzato e componibile delle transazioni.

4. Pattern utilizzati e reificazione

- **Specification Pattern:** Separa la logica del predicato di business finanziario dall'iterazione sulla collezione; ogni specifica atomica rispetta rigorosamente il principio di singola responsabilità (SRP) ed è testabile in completo isolamento.
- **Factory Method / Combiner:** La classe di utilità `TransactionSpecifications` centralizza la creazione e la combinazione booleana (`spec.and(...)`) a partire dal record `TransactionFilterCriteria`.
- **Functional Pipeline e Defensive Copying:** L'applicazione del filtro sfrutta `Stream.filter(spec::isSatisfiedBy)` e restituisce liste congelate e immutabili (`List.copyOf`), proteggendo l'integrità contabile del portafoglio da qualsiasi mutazione esterna.

Capitolo 3

Sviluppo

In questo capitolo vengono presentate le attività di testing automatizzato e le caratteristiche avanzate del linguaggio Java impiegate nello sviluppo dei sottosistemi assegnati.

3.1 Testing Automatizzato

Il progetto adotta la suite **JUnit 5** per garantire l'assenza di regressioni e verificare in modo rigoroso e ripetibile la correttezza della logica applicativa tramite il comando automatizzato:

```
./gradlew check
```

Sono state implementate suite di test unitari e di integrazione esaustive a copertura di tutti i moduli di competenza:

- **Dashboard, Contabilità di Cassa e Transazioni (Favour Emuwahen):**
  - `CashTransactionTest` e `CashTransactionBuilderTest`: verifica della costruzione fluente di `CashTransaction`, validazione delle invarianti di business (es. obbligatorietà di `friendName` per i prestiti) e correttezza dei metodi getter.
  - `CashTransactionFactoryTest`: verifica della generazione automatica di spese, entrate e prestiti con corretta assegnazione del segno algebrico e conformità degli asset monetari.
  - `TransactionTest`: test dei contratti base di uguaglianza (`equals`), codice hash (`hashCode`) e immutabilità della classe sigillata `AbstractTransaction`.
  - `CashAccountTest` e `WalletTest`: verifica dell'integrità del portafoglio monetario, calcolo algebrico esatto del saldo cumulativo e corretta registrazione delle transazioni nello storico.
  - `DefaultBudgetMonitorTest` e `BudgetAlertStrategyTest`: verifica del monitoraggio del budget mensile e reificazione delle strategie di allarme (`SafeBudgetStrategy`, `WarningBudgetStrategy`, `CriticalBudgetStrategy`) con diverse soglie percentuali.
  - `MonthlyInsightTemplateTest` e `DefaultWalletInsightServiceTest`: test dell'algoritmo a template per il confronto statistico mensile, gestione rigorosa della divisione per zero e calcolo dei delta percentuali per spese, entrate e risparmi.
  - `DefaultFriendLoanSummaryServiceTest`: verifica dell'aggregazione dei crediti e debiti tra privati, calcolo dei saldi aperti e tracciamento delle quote restituite.

- `DefaultCategoryServiceTest`: test della ripartizione percentuale delle uscite per categoria e verifica della corretta graduatoria di spesa (*expense breakdown*).
- `DefaultTransactionHistoryFilterServiceTest`: collaudo esaustivo del filtraggio combinato per intervallo di date, categoria, parola chiave e ordinamento cronologico/monetario.
- `DefaultDashboardFacadeTest` e `DefaultDashboardSnapshotTest`: test di integrazione della facciata della dashboard, verifica della consistenza atomica dello snapshot aggregato e isolamento dei dati contabili.
- `DefaultDashboardControllerTest`: verifica del coordinamento MVC, propagazione delle azioni utente dalla vista al modello e test di notifica reattiva come `WalletObserver`.

- **Valute, Conversione, Impostazioni e Temi (Arianna Palmucci):**
  - `CurrencyUnitTest`, `FiatCurrencyTest`, `CryptoCurrencyTest`, `StockMarketCurrencyTest`, `CurrencyTest`: validazione delle categorie valutarie, contratti polimorfici, display decimals e formattazione simbolica.
  - `AssetTest`: test di quantificazione patrimoniale immutabile (`BigDecimal` e `CurrencyUnit`) con arrotondamento esatto.
  - `BasicCurrencyConverterTest` e `CurrencyConverterTest`: verifica della triangolazione cross-currency con dollaro USD e assenza di troncamenti nei cambi.
  - `BankTest`, `BankCalculatorTest`, `BankListTest`, `BankServiceTest`: test del simulatore bancario con spread percentuale e commissioni fisse.
  - `CurrencyAlertTest`, `CurrencyAlertServiceTest`, `MarketAlertTest`, `WatchListTest`: collaudo del monitoraggio tassi, rilevamento allarmi violati e memorizzazione delle coppie preferite.
  - `MockExchangeRateAPITest` e `ExchangeRateAPIClientTest`: test del provider deterministico offline e della resilienza con cache a tempo (TTL).
  - `SettingsTest`, `SettingsSnapshotTest`, `SettingsManagerTest`, `ThemeManagerTest`: test della persistenza JSON, del pattern Memento per lo storico configurazioni e dell'iniezione CSS dinamica.

Tutti i test vengono eseguiti con successo al 100% garantendo l'assenza di regressioni. Il codice rispetta rigorosamente i controlli statici di Checkstyle, PMD e SpotBugs con zero violazioni.

3.2 Note di Sviluppo

3.2.1 Favour Emuwahen

Di seguito viene fornito l'elenco analitico delle feature avanzate del linguaggio Java (versione 21 LTS) e delle librerie applicative impiegate nello sviluppo del sottosistema Dashboard e Contabilità, corredate da specifici riferimenti e permalink al repository GitHub:

- **Gerarchie Sigillate ed Ereditarietà Controllata (`sealed classes`):** Utilizzo di classi astratte sigillate (`public abstract sealed class AbstractTransaction permits CashTransaction, InvestmentTransaction`) per delimitare a compile-time l'albero polimorfico delle transazioni monetarie, abilitando controlli di esaustività ed escludendo estensioni non autorizzate.

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/model/transactions/base/AbstractTransaction.java

- **Progettazione con Interfacce Funzionali e Generici Componibili (`Predicate<T>`):** L'interfaccia di specifica estende `Predicate<CashTransaction>` e implementa metodi di default componibili algebricamente (`and`, `or`, `not`) tramite espressioni lambda.

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/model/dashboard/specification/TransactionSpecific
    java

- **Pattern Matching per Switch e Record Patterns (Java 21):** Utilizzo del pattern matching per switch con decostruzione dei record (`case PaymentSource.NoPaymentChannel() -> ...`) all'interno dei selettori di stato e di pagamento, eliminando cast manuali e variabili locali inutilizzate.

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/view/dashboard/impl/NewTransactionDialog.java

- **Records Immutabili per il Trasferimento Dati (*Data Transfer Objects*):** Definizione di record nativi per modellare lo stato di presentazione della UI (`DashboardViewState`, `HeaderCardState`, `BalanceCardState`, `BudgetCardState`) e le strutture di richiesta (`NewTransactionRequest`, `BudgetStatus`, `WalletInsight`), garantendo immutabilità strutturale e azzerando il codice *boilerplate*.

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/view/dashboard/state/DashboardViewState.java

- **Stream API Avanzate e Pipeline di Riduzione Funzionale (`Collectors.groupingBy`):** Elaborazione dello storico transazioni mediante pipeline con operatori intermedi e terminali (`.filter()`, `.map()`, `.sorted()`, `.reduce()`) per il calcolo dei saldi cumulativi (`BigDecimal.ZERO, BigDecimal::add`) e raggruppamento delle spese con `Collectors.groupingBy()` in `DefaultCategoryService`.

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/model/dashboard/impl/DefaultCategoryService.java

- **Aritmetica Decimale Esatta (`java.math.BigDecimal`) e Arrotondamento Finanziario:** Rappresentazione e calcolo rigoroso di tutti gli importi, percentuali di budget e tassi di variazione mese su mese con `BigDecimal` e arrotondamento bancario controllato (`RoundingMode.HALF_UP`), eliminando gli errori di approssimazione dell'aritmetica floating-point IEEE 754.

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/model/dashboard/impl/AbstractMonthlyInsightTempla
    java

- **Pattern Creazionale Builder Fluente con Validazione delle Invarianti:** Costruzione atomica di oggetti complessi con controllo a tempo di creazione dei vincoli

di consistenza contabile (es. obbligatorietà del campo `friendName` per transazioni di tipo `FRIEND_LOAN`):

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/model/transactions/base/CashTransaction.java

- **Disaccoppiamento ad Eventi con `MessageBus`:** Architettura orientata agli eventi per la sincronizzazione dei componenti attraverso la pubblicazione e sottoscrizione disaccoppiata di eventi applicativi (`TransactionAddedEvent`).

  - *Riferimento*: https://github.com/leoI0II/OOP25-UniBodget/blob/main/app/src/main/java/it/unibo/unibodget/model/utils/MessageBus.java

- **Qualità del Codice e Conformance a Checkstyle (`gradle-java-qa`):** Totale conformità agli standard di stile e design imposti dal plugin `org.danilopianini.gradle-java-qa` con 0 violazioni su Checkstyle, PMD e SpotBugs (eliminazione costanti magiche, copie difensive, assenza di catch generici e documentazione Javadoc).

3.2.2 Arianna Palmucci

Di seguito viene fornito l'elenco puntato delle feature avanzate del linguaggio Java (versione 21 LTS) e delle librerie esterne impiegate nel sottosistema valute, convertitore, impostazioni e temi:

- **Progettazione con Tipi Generici Bounded e Wildcards:** Definizione di vincoli generici bounded (`T extends AbstractTransaction`) e wildcards (`Supplier<? extends AbstractWallet<?»`) per garantire type-safety e riuso nelle pipeline del modello [4, 1].

- **Uso di Lambda Expressions e Interfacce Funzionali:** Disaccoppiamento di azioni e callback asincrone tramite le interfacce standard `Supplier<T>`, `Consumer<T>`, `Function<T, R>` e `BiFunction<T, U, R>` nelle viste e nei controller per una programmazione funzionale dichiarativa [1].

- **Stream API, Pipeline Funzionali e Optional:** Elaborazione dichiarativa delle collezioni di tassi, filtraggio delle allerte e trasformazioni di liste di valute evitando cicli imperativi mutabili, con gestione sicura dei valori assenti tramite `Optional`.

- **Java Time API (`java.time`, JSR-310):** Gestione temporale immutabile con `Instant`, `Duration`, `LocalDate` per la scadenza della cache e l'ordinamento delle serie temporali dei grafici [6].

- **Networking Moderno con Java HTTP Client (`java.net.http`):** Utilizzo di `HttpClient` e `HttpRequest` con supporto nativo HTTP/2 e timeout dedicati per l'interazione RESTful resiliente con i servizi web di cambio valute [5].

- **Serializzazione e Deserializzazione JSON Avanzata con Jackson:** Configurazione personalizzata con moduli dedicati (`SimpleModule`) e custom serializer/deserializer per la deserializzazione polimorfica di `CurrencyUnit` e date ISO 8601.

- **Libreria Apache Commons CSV (`org.apache.commons.csv`):** Generazione ed esportazione conforme di file tabellari standard secondo la specifica RFC 4180 [3].

- **Logging Strutturato con SLF4J / Logback:** Tracciamento diagnostico delle conversioni e degli stati di cache con livelli differenziati (`DEBUG`, `INFO`, `WARN`, `ERROR`).
- **JavaFX (SceneGraph, Charts, CSS Dinamico, Hierarchy Styling):** Realizzazione di grafici storici `LineChart`, iniezione dinamica di classi CSS per la tematizzazione e sfruttamento dell'ereditarietà naturale del motore CSS di JavaFX [7].
- **Adozione dei Design Pattern GoF:** Reificazione rigorosa dei pattern *Strategy*, *Adapter*, *Observer*, *Memento*, *Facade* e *Cache Proxy* [2, 8].

Capitolo 4

Commenti finali

4.1 Autovalutazione e lavori futuri

4.1.1 Favour Emuwahen

All'interno del progetto UniBodget mi sono occupato della progettazione architetturale, dello sviluppo e del collaudo del sottosistema **Dashboard**, della gestione dei **Portafogli di Cassa (*Cash Accounts*)**, del ciclo di vita delle **Transazioni Contanti (*Cash Transactions*)**, del monitoraggio del **Budget**, del motore di analisi statistica dei **Trend Mensili (*Monthly Insights*)** e del motore di filtraggio avanzato con lo **Specification Pattern**.

Punti di forza del contributo:

- **Solidità architetturale e applicazione rigorosa dei design pattern**: L'adozione coordinata di pattern creazionali (*Builder*, *Factory Method*), comportamentali (*Template Method*, *Specification*, *Strategy*) e strutturali (*Facade*, *Passive View*) ha reso il codice modulare, orientato all'estendibilità futura senza violare il principio *Open-Closed* (OCP).
- **Integrità contabile e immutabilità**: Nessuna transazione può essere generata in uno stato parziale o incoerente grazie alle invarianti verificate a tempo di costruzione nel Builder. Tutti i calcoli patrimoniali operano con aritmetica esatta `BigDecimal` e arrotondamento bancario controllato (`RoundingMode.HALF_UP`).
- **Qualità formale del codice e conformità QA**: Conseguimento di **0 violazioni statiche** secondo il severo profilo imposto da `gradle-java-qa` (Checkstyle, PMD e SpotBugs), con totale rimozione di numeri magici di layout, copie difensive e incapsulamento rigoroso.
- **Copertura di test esaustiva**: Realizzazione di una suite di oltre 15 classi di test automatici con JUnit 5 con un tasso di successo del 100%, verificando sia i casi nominali sia le condizioni al contorno (divisione per zero, transazioni prive di categoria, vincoli sui prestiti).

Punti di debolezza ed elementi migliorabili:

- **Sincronizzazione asincrona**: Le operazioni di ricalcolo della dashboard e di salvataggio dello snapshot vengono attualmente eseguite in modo sincrono sul thread

chiamante; l'introduzione di `CompletableFuture` per il recupero concorrente delle statistiche mensili avrebbe ulteriormente ottimizzato i tempi di caricamento su collezioni storiche di dimensioni molto elevate.

- **Conversione multivaluta omnicomprensiva nel cruscotto**: Attualmente la dashboard calcola il saldo aggregato aggregando le transazioni appartenenti al portafoglio nella sua valuta base specifica; una conversione istantanea incrociata omnicomprensiva su tutte le valute mondiali nella vista principale arricchirebbe ulteriormente l'esperienza d'uso.

Ruolo all'interno del gruppo: All'interno del team ho curato la progettazione e l'implementazione del cuore funzionale dell'applicazione: il sottosistema Dashboard, il motore delle transazioni di cassa, il monitoraggio delle soglie di spesa e l'integrazione del sistema di visualizzazione grafica. Ho inoltre coordinato la definizione dei contratti di interfaccia DTO (tramite Java Records) che hanno permesso ai colleghi di lavorare in parallelo senza interferenze sui rispettivi moduli.

Sviluppi futuri: Qualora il progetto venisse proseguito per una distribuzione aperta, le direttrici evolutive più promettenti includono:

1. **Budgeting predittivo intelligente**: Integrazione di modelli statistici o di machine learning leggero per prevedere l'andamento delle spese a fine mese sulla base dello storico dei consumi settimanali.
2. **Esportazione contabile avanzata**: Generazione automatica di rendiconti periodici e prospetti fiscali in formato PDF vettoriale, CSV RFC 4180 ed Excel.
3. **Sincronizzazione Cloud sicura**: Espansione del layer di persistenza locale verso un'architettura ibrida con replica cifrata dei dati su cloud storage personale.

4.1.2 Arianna Palmucci

Il mio ruolo all'interno del progetto ha riguardato la modellazione del dominio polimorfico delle valute, il motore di conversione, l'integrazione con provider di mercato esterni dotati di caching resiliente, il monitoraggio dei tassi con watchlist e allerte, nonché la gestione centralizzata delle impostazioni, dei temi visivi dinamici e della persistenza su file JSON.

Punti di forza. Sono particolarmente soddisfatta dell'elevato livello di astrazione e robustezza raggiunto dal motore di conversione: l'impiego di `BigDecimal` con rounding controllato ha eliminato gli errori di calcolo monetario, mentre l'architettura a Proxy di cache e mock garantisce la totale usabilità dell'applicazione anche in assenza di connessione Internet. Inoltre, l'implementazione del pattern Memento per le preferenze e l'iniezione CSS dinamica a caldo consentono un'esperienza utente moderna e fluida.

Punti di debolezza e difficoltà affrontate. La gestione dell'eterogeneità dei payload JSON restituiti dalle diverse API pubbliche e il rispetto dei severi limiti di frequenza delle chiamate hanno richiesto una complessa attività di normalizzazione e gestione delle eccezioni. Inoltre, la serializzazione polimorfica con Jackson Databind per la gerarchia di `CurrencyUnit` ha comportato una configurazione meticolosa dei moduli personalizzati.

Lavori futuri. Come sviluppi futuri, risulterebbe interessante arricchire il simulatore bancario con l'interrogazione diretta di API bancarie in standard Open Banking (PSD2) e l'estensione della watchlist a un numero illimitato di mercati azionari internazionali.

4.2 Difficoltà Incontrate e Commenti per i Docenti

Nel corso del progetto, il team ha affrontato e superato diverse difficoltà significative:

- **Conciliazione tra JavaFX e Rigorosa Analisi Statica (QA):** L'integrazione del toolkit grafico JavaFX con le severe regole di analisi statica imposte dal plugin del Prof. Pianini (`gradle-java-qa`, comprendente Checkstyle, PMD e SpotBugs) ha richiesto un impegno notevole. In particolare, la gestione delle classi interne anonime, la rimozione sistematica di costanti magiche di layout e la corretta giustificazione dei pattern di esposizione di riferimenti per i nodi grafici hanno reso necessario uno studio approfondito della progettazione ad oggetti.
- **Eliminazione dei Numeri Magici e Token Tipografici:** Rendere l'interfaccia JavaFX dinamica e gradevole rispettando il vincolo di zero numeri magici ha imposto l'estrazione meticolosa di ogni spaziatura, coordinata grafica e raggio di curvatura in classi costanti di utilità visiva.
- **Aritmetica e Precisione Finanziaria:** Garantire la coerenza monetaria tra saldi di cassa, quote azionarie frazionate e percentuali di cambio ha imposto l'adozione pervasiva di `BigDecimal`, prestando massima attenzione alle scale di divisione e alla gestione delle divisioni per zero nei trend statistici.
- **Lavoro Cooperativo e Versionamento con Git:** La suddivisione del lavoro su branch paralleli e la convergenza su classi condivise (come `MainViewController` e i modelli di portafoglio) hanno consolidato le competenze del gruppo nell'uso professionale del controllo di versione, nella risoluzione dei conflitti e nella revisione del codice tramite pull request.

Appendice A

Guida Utente

Questa appendice illustra in modo operativo le funzionalità di **UniBodget**, guidando l'utente attraverso l'esecuzione del software e le principali operazioni finanziarie.

Avvio dell'Applicazione

Per compilare ed eseguire l'applicazione desktop, è sufficiente posizionarsi nella cartella principale del progetto ed eseguire tramite terminale il comando Gradle:

```
./gradlew run
```

Il runtime JavaFX provvederà a caricare le preferenze utente persistite, ripristinare i conti, le transazioni e le impostazioni, e visualizzare la finestra principale dell'applicazione, suddivisa tra la barra laterale di navigazione (*Sidebar*) e l'area centrale, impostata di default sulla **Dashboard**.

Panoramica della Dashboard

La schermata principale della Dashboard è organizzata in aree logiche ad alta densità informativa:

1. **Barra superiore di intestazione**: Mostra il titolo della schermata, il selettore del portafoglio attivo (*Cash Account*) tramite menu a tendina e il pulsante *"New Wallet"* per la creazione di un nuovo portafoglio.
2. **Scheda Patrimonio e Saldo**: Espone a caratteri cubitali il saldo contabile totale del conto attivo, formattato nella divisa di riferimento (es. EUR, USD).
3. **Riepilogo Trend Mensili (*Monthly Insights*)**: Tre schede comparative che espongono in tempo reale il confronto rispetto al mese precedente per:
   - *Risparmi netti*: Differenza tra entrate e uscite con indicazione del delta percentuale.
   - *Uscite totali*: Somma delle spese del mese con badge cromatico di tendenza.
   - *Entrate totali*: Totale dei ricavi incassati rispetto al mese antecedente.
4. **Sezione Monitoraggio Budget**: Indicatore grafico circolare che riporta la percentuale di budget consumata, accompagnata da un badge cromatico:

- *Verde (`SAFE`)*: Spese ampiamente al di sotto della soglia limite.
- *Giallo/Arancio (`WARNING`)*: Superamento della soglia percentuale di pre-allarme (es. 80%).
- *Rosso (`CRITICAL`)*: Saturazione o superamento del 100% del limite prefissato.

Accanto all'indicatore è presente il pulsante di modifica rapida per aggiornare il limite massimo e la soglia percentuale.

5. **Grafico Ripartizione Spese**: Diagramma circolare interattivo (*PieChart*) con tooltip dinamici che scompone le uscite per voce di costo (Alimentari, Affitto, Trasporti, Svago, ecc.).

6. **Tabella Cronologia Movimenti**: Prospetto analitico con colonne per Data, Descrizione, Categoria, Importo monetario, Tipologia e pulsante di cancellazione rapida.

Operazioni Guidate

Inserimento di una Nuova Transazione

1. Premere il pulsante *"+ **New Transaction**"* situato nella schermata della Dashboard.
2. Selezionare la *Tipologia* contabile desiderata (*Expense*, *Income*, *Friend Loan*).
3. Inserire l'importo monetario (il sistema valida automaticamente cifre, virgole e punti decimali).
4. Selezionare la categoria semantica associata e specificare la data tramite il calendario integrato.
5. Inserire una causale descrittiva e note facoltative.
6. Se si registra un prestito informale (*Friend Loan*), specificare obbligatoriamente il nominativo dell'amico oppure selezionare un prestito pregresso per registrarne la restituzione.
7. Premere ***"Save Transaction"***. Il saldo, i grafici e il budget si aggiorneranno istantaneamente in modo atomico.

Filtraggio Avanzato della Cronologia

1. Impostare l'intervallo temporale tramite i selettori di data iniziale e finale.
2. Selezionare un filtro per tipologia di movimento o per categoria dal rispettivo menu a tendina.
3. Digitare una parola chiave nel campo di ricerca testuale (la ricerca opera in modalità *case-insensitive* su descrizione e note).
4. L'ordinamento può essere commutato a piacere (per data o per importo).
5. Cliccando su ***"Export CSV"***, la cronologia filtrata viene esportata su un file tabellare conforme allo standard RFC 4180.

Convertitore Valute e Impostazioni di Sistema

1. **Conversione Istantanea**: Nella sezione *Currency Converter*, inserire l'importo, selezionare le valute e premere *"Convert"*. Il pulsante *"Swap"* inverte istantaneamente le valute.

2. **Simulatore Bancario**: Selezionare la banca desiderata per visualizzare lo spread commerciale e le commissioni fisse applicate rispetto al tasso reale di mercato.

3. **Personalizzazione Temi**: Nella schermata *Settings*, commutare istantaneamente il tema visivo tra Dark e Light, selezionare la famiglia tipografica preferita e la valuta base globale, con anteprima dal vivo immediata.

Bibliografia

[1] Joshua Bloch. *Effective Java (Third Edition)*. Addison-Wesley Professional, 2018.

[2] Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.

[3] Y. Shafranovich. *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180, 2005.

[4] Oracle Corporation. *Lesson: Generics (Updated to Java SE 21)*. https://docs.oracle.com/javase/tutorial/java/generics/

[5] Oracle Corporation. *Package java.net.http Specification (Java SE 21)*. https://docs.oracle.com/en/java/javase/21/docs/api/java.net.http/package-summary.html

[6] Oracle Corporation. *Trail: Date Time (Java SE Tutorial, JSR 310)*. https://docs.oracle.com/javase/tutorial/datetime/

[7] OpenJFX Project. *JavaFX CSS Reference Guide*. https://openjfx.io/javadoc/21/javafx.graphics/javafx/scene/doc-files/cssref.html

[8] Alexander Shvets. *Design Patterns in Java*. Refactoring.Guru, https://refactoring.guru/design-patterns/java